

reviews.csv – Data Exploration Report

Findings from the exploration pass on `reviews.csv` : structural profile → null/missingness pass → duplicate pass → referential integrity → value-range sanity → candidate questions. Full working evidence is in `notebooks/exploration_reviews.ipynb`.

Structural Profile

- Shape: 6,327 rows × 23 columns.
- All dtypes look correct at face value (`rating` / `sentiment_score` as float, `verifiedPurchase` as bool, ids/text as str). No silent-mistype issues like a price column read as text – worth confirming again once `products.csv` is profiled, since that file has more numeric-looking columns at higher risk of this.

Null / Missingness Pass

Column(s)	Null rate
<code>images/0</code> → <code>images/7</code>	91.8% → 99.9%, increasing
<code>videos/0</code>	99.5%
<code>productVariant</code>	7.8%
<code>cleaned_review_text</code>	0.6%
<code>reviewText</code>	0.6%
<code>reviewMetadata</code>	0.5%
<code>reviewTitle</code>	0.2%
<code>rating</code>	0.1% (7 rows)

Image/video columns: null rate climbs from ~92% (`images/0`) to ~99.9% (`images/7`) – expected shape, since most reviews attach zero photos and a shrinking few attach more. Not a data quality problem; it's the natural distribution of how many photos a reviewer bothers to attach. Confirmed every populated link resolves to a real-looking URL rather than garbage.

All 927 populated image links end in `.jpg` – clean, no truncation issue on this column.

Real finding, distinct from "missing": 3 of 34 populated `videos/0` links ($\approx 9\%$) are **truncated URLs**, not nulls — e.g. `.../E10` instead of a full `.mp4` link. This is corruption within populated data, not absence of data, and needs its own handling decision in the Transformer (drop the row's video field, or flag it), separate from the $\sim 99\%$ that are legitimately null.

`reviewMetadata` (location + date string) is null for $\sim 0.5\%$ of rows. Spot check: these rows still have real `reviewText` / `rating` — this looks like an isolated scraping gap on that one field, not a sign the whole row is bad. Safe to keep the row; `reviewMetadata` may be treated later as nullable.

Pattern found: all 7 rows with a null `rating` also have null `reviewText`, and all 7 have `sentiment_score == 0.0` exactly. Checked next whether `sentiment_score = 0.0` is a genuine "neutral" computed score, or simply a default value the scraper/pipeline fills in when there's no text to score at all.

Investigated further: `sentiment_score == 0.0` occurs on 251 rows total, but only 38 of those have null `reviewText` — the other 213 have real review text and still score exactly `0.0`. So `0.0` is not purely a null-text default; it's more likely a genuine possible output of the sentiment model that happens to coincide with the default value. This means an earlier hypothesis (treat all `0.0` as "unknown" rather than "neutral") does not hold as a blanket rule — the 38 null-text rows are still worth flagging as likely-default, but the other 213 should be trusted as real scores. Worth reflecting this nuance in the Transformer design later.

38 rows have null `reviewText` but a present `sentiment_score` (all `0.0` in this subset).

Duplicate Pass

- Full-row duplicates: 0
- `reviewID` duplicates: 0 — fully unique across all 6,327 rows.

`reviewID` is a clean, fully unique key — safe to use as the natural primary key / `_id` source for a `reviews` collection.

Referential Integrity (against `products.csv`)

Deferred to the joint exploration once `products.csv` is profiled. `productASIN` currently shows 700 distinct products referenced by at least one review.

Value-Range / Sanity Pass

`rating` ranges 1.0–5.0 as expected, always a whole number in practice (no genuine half-star values observed) despite being stored as float64 – that will need casting in the Transformer design.

`verifiedPurchase` : clean boolean, 6,167 `True` / 160 `False` .

Discrepancy from the dataset's own listing: the Kaggle description states

`sentiment_score` ranges 0–1, but the actual computed range is **-1.0 to 1.0** – worth flagging and revising other things where needed.

`reviewPosition` is a clean 1–10 ranking (position of the review within its product's review list). Manual verification found these positions are not quite correct and this column may not be needed anymore since it's stale data.

Real finding from manual spot-checking (not caught by automated stats): at least one row contains a non-English review (Hindi) where `cleaned_review_text` is null despite `reviewText` having real content – meaning the cleaning/sentiment pipeline that produced this dataset likely only processes English text, silently leaving `cleaned_review_text` null and `sentiment_score` at a default (`0.0`) for non-English rows rather than actually scoring them. This connects to the `sentiment_score == 0.0` investigation above – likely the same root cause (no clean text to score) showing up two different ways: missing text entirely, or non-English text the cleaning step couldn't handle.

Column-Specific Notes

`productVariant` is a semi-structured free-text field packing multiple attributes (Size/Color/Height/Weight) into one string with no consistent delimiter or field order between rows – will need parsing logic in the Transformer if any of these sub-attributes are wanted individually, not just a null/not-null check. ~7.8% null, consistent with products that simply don't have variants.

`reviewMetadata` packs a location and a date into one string (`"Reviewed in <location> on <date>"`) with no separator – will need parsing (regex or split) to extract structured `location` and `date` fields in the Transformer, rather than keeping this as one opaque string.

Candidate Questions

- Does `verifiedPurchase` correlate with `rating` or `sentiment_score`?
- Does having attached images/video correlate with `helpfulVoteCount`?
- How does `sentiment_score` distribution differ between English and non-English reviews (once that's identifiable – see finding above)?
- Which products have the widest spread between `rating` and `sentiment_score` (i.e. star rating and text sentiment disagree)?
- Does `reviewPosition` (1-10) correlate with `helpfulVoteCount` – do earlier-shown reviews
get more helpful votes regardless of content (regardless of the stale data in that column)?